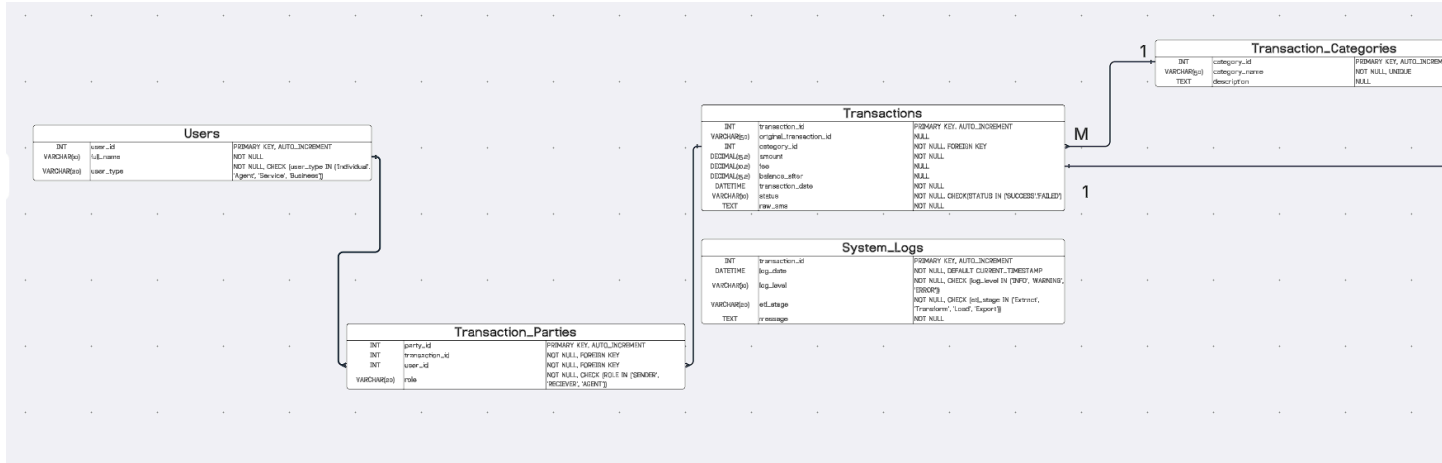


ERD Design



1. Entity Relationship Diagram (ERD) Documentation

The ERD for this project is maintained as a separate diagram file (docs/erd_diagram.png) in the repository. This section documents the entities, their relationships, and the design decisions behind them.

1.1 Entities Overview

The database consists of five entities designed from analyzing 1,691 real MoMo SMS transaction records. Each entity was chosen to represent a distinct concept in the MoMo transaction ecosystem.

Entity	Purpose	Record Count
Users	Stores all parties involved in transactions: individuals, agents, businesses, and services	Approximately 15-20 unique users
Transaction_Categories	Reference table defining the nine types of MoMo transactions	9 fixed categories
Transactions	Main table storing each processed MoMo SMS transaction	Up to 1,661 transaction records
Transaction_Parties	Junction table linking users to transactions with their roles	2-3 records per transaction
System_Logs	Tracks ETL pipeline activity for debugging and auditing	Variable, depends on pipeline runs

1.2 Relationships

Transaction_Categories (1) to Transactions (M): One category can have many transactions, but each transaction belongs to exactly one category. For example, the Payment category contains 662 transactions while the Withdrawal category contains only 3.

Users (M) to Transactions (M) via Transaction_Parties: This is a many-to-many relationship resolved through the Transaction_Parties junction table. One user participates in many transactions, and one transaction involves multiple users. Each record in Transaction_Parties includes a role column (Sender, Receiver, or Agent) to distinguish how each user is involved.

System_Logs (Standalone): This table has no foreign key relationships. It operates independently to record ETL pipeline activity without being tied to specific transactions or users.

2. Data Dictionary

This section provides a complete description of every table and column in the database schema.

2.1 Users Table

Stores all parties involved in MoMo transactions. Users are identified by name since phone numbers in the dataset are shared across multiple people and cannot serve as reliable identifiers.

Column	Data Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique identifier for each user in the system.
full_name	VARCHAR(100)	NOT NULL	Full name of the user as extracted from the SMS data. Examples: Jane Smith, Agent Sophia, DIRECT PAYMENT LTD, Airtime.
user_type	VARCHAR(20)	NOT NULL, CHECK	Classifies the user. Individual for people like Jane Smith. Agent for cash-out agents like Agent Sophia. Business for companies like DIRECT PAYMENT LTD. Service for automated services like Airtime and WASAC.

2.2 Transaction_Categories Table

A small reference table defining the nine types of MoMo transactions identified from analyzing the SMS data patterns. Each category is determined by specific keywords or prefix codes in the SMS body.

Column	Data Type	Constraints	Description
category_id	INT	PK, AUTO_INCREMENT	Unique identifier for each transaction category.
category_name	VARCHAR(50)	NOT NULL, UNIQUE	Name of the category. One of: Payment, Transfer, Bank Deposit, Incoming Money,

			Airtime/Bill Payment, Third-Party Service, Withdrawal, Bank Transfer, Reversal.
description	TEXT	NULL	Brief explanation of what this category represents, including how it is identified from the SMS body.

2.3 Transactions Table

The main table storing each processed MoMo SMS transaction. Uses an auto-generated primary key because approximately 50% of transactions in the data (all transfers and all bank deposits) do not have a MoMo-issued transaction ID.

Column	Data Type	Constraints	Description
transaction_id	INT	PK, AUTO_INCREMENT	Internal unique identifier for each transaction, auto-generated by the database.
original_transaction_id	VARCHAR(50)	NULL	The original transaction ID from the MoMo SMS when available. Appears as TxId for payments and airtime, or Financial Transaction Id for incoming money and third-party services. NULL for transfers and bank deposits which have no ID in the SMS.
category_id	INT	NOT NULL, FK	Foreign key linking to the Transaction_Categories table. Identifies what type of transaction this is.
amount	DECIMAL(15,2)	NOT NULL	Transaction amount in Rwandan Francs (RWF). Parsed from formats like 1,000 RWF or 2000 RWF in the SMS body and stored as a clean number.
fee	DECIMAL(10,2)	NULL	Fee charged for the transaction in RWF. Ranges from 0 to 8,000 in the dataset. NULL for transaction types that do not report a fee, such as bank deposits and incoming money.
balance_after	DECIMAL(15,2)	NULL	Account balance after the transaction was completed. NULL when the SMS does not include balance information.
transaction_date	DATETIME	NOT NULL	Date and time when the transaction occurred, extracted from the SMS body in the format YYYY-MM-DD HH:MM:SS.
status	VARCHAR(10)	NOT NULL, CHECK	Whether the transaction completed successfully or failed. Constrained to Success or Failed. Derived from *S* (Success) or *R* (Failed) indicators in the SMS body.
raw_sms	TEXT	NOT NULL	The complete original SMS body text preserved for reference and debugging. This is an

			intentional denormalization to allow re-parsing if needed.
--	--	--	--

2.4 Transaction_Parties Table (Junction Table)

Resolves the many-to-many relationship between Users and Transactions. Each record represents one user's involvement in one transaction, with a role specifying how they participated. Most transactions have two parties (sender and receiver), withdrawals include an agent, and bank deposits have only a receiver.

Column	Data Type	Constraints	Description
party_id	INT	PK, AUTO_INCREMENT	Unique identifier for each party record.
transaction_id	INT	NOT NULL, FK	Foreign key linking to the Transactions table. Identifies which transaction this user participated in.
user_id	INT	NOT NULL, FK	Foreign key linking to the Users table. Identifies which user participated in this transaction.
role	VARCHAR(20)	NOT NULL, CHECK	The role this user played in the transaction. Constrained to Sender (person who sent money), Receiver (person who received money), or Agent (cash-out agent involved in withdrawal transactions).

2.5 System_Logs Table

Records ETL pipeline activity for debugging and auditing. This table is standalone with no foreign key relationships to other tables. Log entries are written by the Python ETL code at key checkpoints during data processing.

Column	Data Type	Constraints	Description
log_id	INT	PK, AUTO_INCREMENT	Unique identifier for each log entry.
log_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when the log entry was created. Automatically set to the current time by the database.
log_level	VARCHAR(10)	NOT NULL, CHECK	Severity of the log entry. INFO for normal operations like successfully parsing records. WARNING for non-fatal issues like skipping OTP

			messages. ERROR for failures like being unable to parse an amount from an SMS.
etl_stage	VARCHAR(20)	NOT NULL, CHECK	Which step of the ETL pipeline generated this log. Extract for XML parsing (parse_xml.py). Transform for cleaning and categorizing (clean_normalize.py, categorize.py). Load for database insertion (load_db.py). Export for JSON generation (run.py).
message	TEXT	NOT NULL	Human-readable description of what happened, written by the Python ETL code at key checkpoints during the pipeline.

3. Indexes for Performance Optimization

The following indexes are created to optimize query performance on frequently searched columns.

Index Name	Table.Column	Purpose
idx_users_full_name	Users.full_name	Speeds up user lookups by name during ETL processing to check if a user already exists before inserting.
idx_transactions_date	Transactions.transaction_date	Optimizes date range queries for dashboard analytics such as monthly totals and trends.
idx_transaction_category	Transactions.category_id	Speeds up filtering transactions by category for charts like category distribution pie charts.
idx_transactions_status	Transactions.status	Optimizes queries filtering by Success or Failed status.
idx_original_txid	Transactions.original_transaction_id	Enables fast lookup by MoMo transaction ID for deduplication during ETL upsert operations.
idx_party_transaction	Transaction_Parties.transaction_id	Speeds up joining transaction parties when building complete transaction views with user details.
idx_party_user	Transaction_Parties.user_id	Optimizes queries finding all transactions a specific user participated in.
idx_party_role	Transaction_Parties.role	Speeds up filtering by role when analyzing sender vs receiver patterns.
idx_logs_level	System_Logs.log_level	Enables quick filtering to find only ERROR or WARNING entries during debugging.

idx_logs_stage	System_Logs.etl_stage	Speeds up filtering logs by pipeline stage to isolate issues in Extract, Transform, Load, or Export.
idx_logs_date	System_Logs.log_date	Optimizes queries filtering logs by time range to review recent pipeline activity.

4. Constraints Summary

The following constraints enforce data integrity and business rules across the database.

Constraint Name	Table	Type	Rule
PRIMARY KEY	All tables	Primary Key	Each table has an INT AUTO_INCREMENT primary key ensuring every row is uniquely identifiable.
fk_category	Transactions	Foreign Key	category_id must reference a valid category_id in Transaction_Categories.
fk_party_transaction	Transaction_Parties	Foreign Key	transaction_id must reference a valid transaction_id in Transactions.
fk_party_user	Transaction_Parties	Foreign Key	user_id must reference a valid user_id in Users.
chk_user_type	Users	CHECK	user_type must be one of: Individual, Agent, Service, Business.
chk_status	Transactions	CHECK	status must be one of: Success, Failed.
chk_role	Transaction_Parties	CHECK	role must be one of: Sender, Receiver, Agent.
chk_log_level	System_Logs	CHECK	log_level must be one of: INFO, WARNING, ERROR.
chk_etl_stage	System_Logs	CHECK	etl_stage must be one of: Extract, Transform, Load, Export.
UNIQUE	Transaction_Categories	Unique	category_name must be unique across all rows to prevent duplicate categories.

design rationale

Our database schema was designed by first analyzing the actual MoMo SMS data to identify real patterns rather than making assumptions about what the data might contain.

We chose five entities. The **Users** table stores all parties involved in transactions, whether individuals like Jane Smith, agents like Agent Sophia, businesses like DIRECT PAYMENT LTD, or services like Airtime. Each user is identified by an auto-generated user_id as the primary key. We considered using phone numbers as identifiers but discovered that the phone numbers in our dataset are shared across multiple people, making them unreliable. The user_type column classifies each user into one of four categories based on how they appear in the SMS data.

The **Transaction_Categories** table is a small reference table holding our nine transaction types such as Payment, Transfer and Bank Deposit. We kept it minimal with just a name and description after removing direction and service_code during normalization, since both were fully determined by the category name, which violated 3NF.

The **Transactions** table uses an auto-generated primary key because 50% of transactions in the data, specifically all transfers and bank deposits, have no MoMo-issued ID. The original MoMo ID is stored separately in a nullable column for transactions that do have one. We also removed the message and sms_date columns after discovering that message fields were consistently empty across the dataset, and sms_date was always exactly two hours behind transaction_date, a timezone offset providing no new information.

The **Transaction_Parties** junction table resolves the many-to-many relationship between Users and Transactions. Rather than putting sender_id and receiver_id directly on the Transactions table, this approach handles variable participant counts. Withdrawals involve an agent, bank deposits have only a receiver, and most other transactions have two parties.

System_Logs stands alone with no foreign keys, tracking ETL pipeline activity independently from the transaction data it processes.

sql-json mapping

Our SQL tables map to JSON representations following a consistent pattern where each table becomes an array of objects, each row becomes an object, and each column becomes a key-value pair within that object.

Data type mapping follows straightforward rules. SQL INT and DECIMAL columns become JSON numbers. INT AUTO_INCREMENT primary keys like `user_id` and `transaction_id` map directly to integer values. DECIMAL columns like `amount` and `fee` map to floating point numbers with two decimal places, so an amount stored as DECIMAL(15,2) in SQL appears as 2000.00 in JSON. SQL VARCHAR and TEXT columns become JSON strings. DATETIME values are serialized as ISO 8601 formatted strings, for example "2024-05-10T16:31:39" instead of the SQL format "2024-05-10 16:31:39". SQL NULL values, which appear in nullable columns like `original_transaction_id`, `fee` and `balance_after`, are represented as JSON null.

Foreign key handling differs depending on the use case. In flat representations meant for simple data transfer, foreign keys remain as integer IDs. For example, a transaction stores `category_id` as the number 1. In nested API responses meant for frontend consumption, foreign keys are replaced by the full related object. So instead of `category_id: 1`, the response embeds the complete category object with its name and description directly inside the transaction.

Relationship serialization follows the nesting principle. The one-to-many relationship between `Transaction_Categories` and `Transactions` means each transaction contains its category data as a nested object. The many-to-many relationship between `Users` and `Transactions`, resolved through the `Transaction_Parties` junction table in SQL, is serialized as a `parties` array nested inside each transaction object. Each element in that array contains the role (Sender, Receiver or Agent) along with the full user object. This eliminates the need for the junction table structure in JSON since the relationship is expressed naturally through nesting.

`System_Logs` maps independently with no nesting since it has no foreign key relationships to other tables.